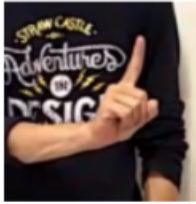
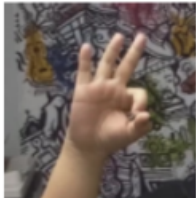
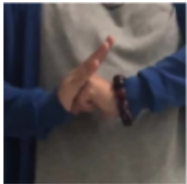
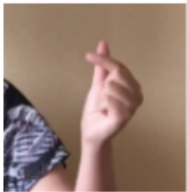
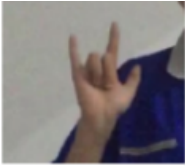
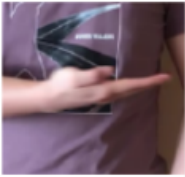
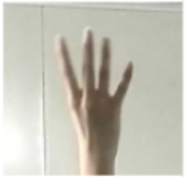
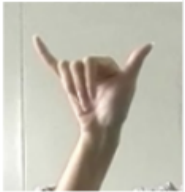
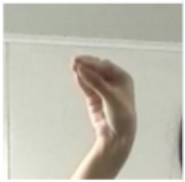
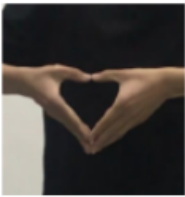
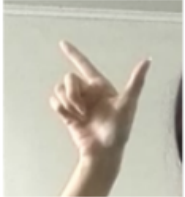
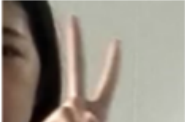
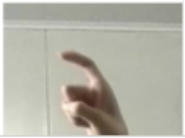


Gesture Recognition Stack Blocks

Gesture Recognition Description

1. The gesture recognition used in this example is based on Baidu Intelligent Cloud platform services, which provide 50,000 free uses per day for learning purposes only. Do not use for commercial purposes. If you need long-term use, please purchase the relevant service. Our company is not responsible for it.
2. For KEY application instructions and API documentation, please refer to the previous course content.
3. Gestures supported by gesture recognition and example images

Serial number	Gestures name	Sample
1	number_1	
2	number_5	
3	fist	
4	ok	
5	pray	
6	congratulation	
7	honour	
8	heart_single	
9	thumb_up	
10	thumb_down	

11	i_love_you		17	number_3	
12	palm_up		18	number_4	
13	heart_1		19	number_6	
14	heart_2		20	number_7	
15	heart_3		21	number_8	
16	number_2		22	number_9	

23	rock	
----	------	--

Experiment Setup

Place yellow blocks in the yellow area, red blocks in the red area, green blocks in the green area, and blue blocks in the blue area according to the experiment setup positions.



Code Content

Code path: /home/yahboom/Dofbot/6.AI_Visual/2.gesture_stack.ipynb

```
#bgr8 to jpeg format
import enum
import cv2
def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#Import related modules

import threading
import time
from Arm_Lib import Arm_Device

# Create robotic arm object
Arm = Arm_Device()
time.sleep(.1)
```

```
# Define gesture recognition function part
import cv2

import time

import demjson
```

```

import pygame

from aip import AipBodyAnalysis
from aip import AipSpeech
from PIL import Image, ImageDraw, ImageFont
import numpy
import ipywidgets.widgets as widgets

# For specific gestures, please refer to the official documentation
https://ai.baidu.com/ai-doc/BODY/4k3cpywrv

hand={'One':'Number 1','Two':'Number 2','Three':'Number 3','Four':'Number 4',

      'Five':'Number 5', 'Six':'Number 6','Seven':'Number 7',

      'Eight':'Number 8','Nine':'Number 9','Fist':'Fist','Ok':'OK',

      'Prayer':'Prayer','Congratulation':'Respectful
greeting','Honour':'Farewell',

      'Heart_single':'Heart gesture','Thumb_up':'Thumb up','Thumb_down':'Thumb
down',

      'ILY':'I love you','Palm_up':'Palm up','Heart_1':'Double heart 1',

      'Heart_2':'Double heart 2','Heart_3':'Double heart 3','Rock':'Rock',

      'Insult':'Middle finger','Face':'Face'}

# The following keys need to be replaced with your own

""" Body Analysis APPID AK SK """

APP_ID = '18550528'

API_KEY = 'K6PWqtiUTKYK1fYaz1308E3i'

SECRET_KEY = 'IDBUII1j6srF1XVNDX32I2WpuWBwczzk'

client = AipBodyAnalysis(APP_ID, API_KEY, SECRET_KEY)

g_camera = cv2.VideoCapture(0)
g_camera.set(3, 640)
g_camera.set(4, 480)
g_camera.set(5, 30) #Set frame rate
g_camera.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
g_camera.set(cv2.CAP_PROP_BRIGHTNESS, 40) #Set brightness -64 - 64 0.0
g_camera.set(cv2.CAP_PROP_CONTRAST, 50) #Set contrast -64 - 64 2.0
g_camera.set(cv2.CAP_PROP_EXPOSURE, 156) #Set exposure value 1.0 - 5000 156.0

ret, frame = g_camera.read()

```

```
# Define camera display component

image_widget = widgets.Image(format='jpeg', width=600, height=500) #Set
camera display component

display(image_widget)

image_widget.value = bgr8_to_jpeg(frame)
```

```
# Define function to convert and display Chinese text

def cv2ImgAddText(img, text, left, top, textColor=(0, 255, 0), textSize=20):

    if (isinstance(img, numpy.ndarray)): # Check if it's an OpenCV image type

        img = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

        # Create an object that can draw on the given image

        draw = ImageDraw.Draw(img)

        # Font format

        fontStyle = ImageFont.truetype(

            "simhei.ttf", textSize, encoding="utf-8")

        # Draw text

        draw.text((left, top), text, textColor, font=fontStyle)

        # Convert back to OpenCV format

        return cv2.cvtColor(numpy.asarray(img), cv2.COLOR_RGB2BGR)
```

```
# Define variable parameters for different positions
look_at = [90, 164, 18, 0, 90, 90]
p_top = [90, 80, 50, 50, 270]

p_Yellow = [65, 22, 64, 56, 270]
p_Red = [118, 19, 66, 56, 270]

p_Green = [136, 66, 20, 29, 270]
p_Blue = [44, 66, 20, 28, 270]

p_layer_4 = [90, 76, 40, 17, 270]
p_layer_3 = [90, 65, 44, 17, 270]
p_layer_2 = [90, 65, 25, 36, 270]
p_layer_1 = [90, 48, 35, 30, 270]

p_push_over_1 = [90, 90, 5, 0, 90, 150]
p_push_over_2 = [90, 90, 0, 50, 90, 150]

#Define the state of grasping blocks
yellow_grabbed = 0
red_grabbed = 0
```

```
green_grabbed    = 0
blue_grabbed     = 0
```

```
#Define gesture recognition count
```

```
Count_One       = 0
Count_Two       = 0
Count_Three     = 0
Count_Four      = 0
Count_Fist      = 0
```

```
# Define function to move robotic arm, control servos 1-6 simultaneously, p=[s1,s2,s3,s4,s5,s6]
```

```
def arm_move_6(p, s_time = 500):

    for i in range(6):

        id = i + 1

        Arm.Arm_serial_servo_write(id, p[i], s_time)

        time.sleep(.01)

    time.sleep(s_time/1000)
```

```
# Define function to move robotic arm, control servos 1-5 simultaneously, p=[s1,s2,s3,s4,s5]
```

```
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1

        if id == 5:

            time.sleep(.1)

            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))

        elif id == 1 :

            Arm.Arm_serial_servo_write(id, p[i], int(3*s_time/4))
        else:
            Arm.Arm_serial_servo_write(id, p[i], int(s_time))
            time.sleep(.01)
    time.sleep(s_time/1000)
```

```
# Define function to grip blocks, enable=1: grip, =0: release
```

```
def arm_clamp_block(enable):
    if enable == 0:
        Arm.Arm_serial_servo_write(6, 60, 400)
```

```
else:
    Arm.Arm_serial_servo_write(6, 130, 400)

time.sleep(.5)
```

#Number function definition

```
def number_action(index):

    if index == 1:

        # Grab yellow block

        arm_move(p_top, 1000)

        arm_move(p_Yellow, 1000)

        arm_clamp_block(1)

    #         time.sleep(.5)

        arm_move(p_top, 1000)

    elif index == 2:

        # Grab red block

        arm_move(p_top, 1000)

        arm_move(p_Red, 1000)

        arm_clamp_block(1)

        arm_move(p_top, 1000)

    elif index == 3:

        # Grab green block

        arm_move(p_top, 1000)

        arm_move(p_Green, 1000)

        arm_clamp_block(1)

        arm_move(p_top, 1000)

    elif index == 4:

        # Grab blue block

        arm_move(p_top, 1000)

        arm_move(p_Blue, 1000)

        arm_clamp_block(1)
```

```
arm_move(p_top, 1000)

def put_down_block(layer):

    if layer == 1:

        arm_move(p_layer_1, 1000)

        arm_clamp_block(0)

        arm_move_6(look_at, 1000)

    elif layer == 2:

        arm_move(p_layer_2, 1000)

        arm_clamp_block(0)

        arm_move_6(look_at, 1000)

    elif layer == 3:

        arm_move(p_layer_3, 1000)

        arm_clamp_block(0)

        arm_move_6(look_at, 1000)

    elif layer == 4:

        arm_move(p_layer_4, 1000)

        time.sleep(.1)

        arm_clamp_block(0)

        arm_move_6(look_at, 1000)

# Push over blocks

def push_over_block():

    arm_move_6(p_push_over_1, 1000)

    time.sleep(.2)

    arm_move_6(p_push_over_2, 1000)

    time.sleep(.1)
```

```
arm_move_6(look_at, 1000)

time.sleep(1)

global g_layer

g_layer = 0
```

#Move the robotic arm to the position where the camera looks forward

```
arm_move_6(look_at, 1000)

time.sleep(1)
```

```
global g_state_arm

g_state_arm = 0

global g_layer

g_layer = 0

def ctrl_arm_move(index):

    global g_layer

    g_layer = g_layer + 1

    if g_layer >= 5:

        g_layer = 1

    arm_clamp_block(0)

    if index == 1:

        number_action(index)

        put_down_block(g_layer)

    elif index == 2:

        number_action(index)

        put_down_block(g_layer)

    elif index == 3:

        number_action(index)

        put_down_block(g_layer)

    elif index == 4:

        number_action(index)
```

```

        put_down_block(g_layer)

    elif index == 5:

        time.sleep(1)

        push_over_block()

global g_state_arm

g_state_arm = 0

```

```

def start_move_arm(index):
    # Start robotic arm control thread
    global g_state_arm

    if g_state_arm == 0:

        closeTid = threading.Thread(target = ctrl_arm_move, args = [index])

        closeTid.setDaemon(True)

        closeTid.start()

        g_state_arm = 1

```

```

try:

    Arm.Arm_Buzzer_On(1)

    s_time = 300

    Arm.Arm_serial_servo_write(4, 10, s_time)

    time.sleep(s_time/1000)

    Arm.Arm_serial_servo_write(4, 0, s_time)

    time.sleep(s_time/1000)

    Arm.Arm_serial_servo_write(4, 10, s_time)

    time.sleep(s_time/1000)

    Arm.Arm_serial_servo_write(4, 0, s_time)

    time.sleep(s_time/1000)

    while True:

        """1.Take photo """

        ret, frame = g_camera.read()

```

```

""" 2.Call gesture recognition """

raw = str(client.gesture(image_widget.value))

text = demjson.decode(raw)

try:

    res = text['result'][0]['classname']

except:

    # print('Recognition result: Nothing recognized~' )

    # img = cv2ImgAddText(frame, "Not recognized", 250, 30, (0, 0 ,
255), 30)

    img = frame

else:

    # print('Recognition result: ' + hand[res])

    # img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)

    if res == 'One':

        Count_One = Count_One + 1

        Count_Two = 0

        Count_Three = 0

        Count_Four = 0

        Count_Fist = 0

        if Count_One >= 3:

            print('Recognition result: ' + hand[res])

            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 ,
0), 30)

            if yellow_grabbed == 0:

                start_move_arm(1)

            # global yellow_grabbed

            yellow_grabbed = 1

        elif res == 'Two':

            Count_Two = Count_Two + 1

```

```

Count_Three = 0

Count_Four = 0

Count_Fist = 0

if Count_Two >= 3:

    print('Recognition result: ' + hand[res])

    img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 ,
0), 30)

    if red_grabbed == 0:

        start_move_arm(2)

#                global red_grabbed

        red_grabbed = 1

elif res == 'Three':

    Count_Three = Count_Three + 1

    Count_Two = 0

    Count_Four = 0

    Count_Fist = 0

    if Count_Three >= 3:

        print('Recognition result: ' + hand[res])

        img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 ,
0), 30)

        if green_grabbed == 0:

            start_move_arm(3)

#                global green_grabbed

            green_grabbed = 1

elif res == 'Four':

    Count_Four = Count_Four + 1

    Count_Two = 0

    Count_Three = 0

    Count_Fist = 0

    if Count_Four >= 3:

```

```

        print('Recognition result: ' + hand[res])

        img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 ,
0), 30)

        if blue_grabbed == 0:

            start_move_arm(4)

#                global blue_grabbed

            blue_grabbed = 1

    elif res == 'Fist': #Fist

        Count_Fist = Count_Fist + 1

        Count_One = 0

        Count_Two = 0

        Count_Three = 0

        Count_Four = 0

        if Count_Fist >= 3:

            print('Recognition result: ' + hand[res])

            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 ,
0), 30)

            start_move_arm(5)

            yellow_grabbed = 0

            red_grabbed = 0

            green_grabbed = 0

            blue_grabbed = 0

            Count_Fist = 0
        else:
            img = frame

    image_widget.value = bgr8_to_jpeg(img)

except KeyboardInterrupt:

    print(" Program closed! ")

    pass

```

If the set gesture is recognized, the robotic arm will perform the corresponding action. When the robotic arm recognizes a number for the first time, it grips the block and places it on the first layer. When it recognizes a gesture number for the second time, it grips the block and places it on the second layer. When it recognizes a gesture number for the third time, it grips the block and places it on the third layer. When it recognizes a gesture number for the fourth time, it grips the block and places it on the fourth layer. Each time, the color of the block gripped by the robotic arm is determined by the recognized number. Number 1 means yellow, number 2 means red, number 3 means green, number 4 means yellow. If the same number is recognized again, it won't grab unless a fist is recognized, which will push over all blocks, clear the records, and restart.

The gesture and action correspondence for this example is as follows:

- | Number 1 | Grab yellow block from yellow area |
- | Number 2 | Grab red block from red area |
- | Number 3 | Grab green block from green area |
- | Number 4 | Grab blue block from blue area |
- | Fist | Push over stacked blocks, clear records |